

i Forside

Eksamensoppgave i TDT4160 Datamaskiner

Dato: 11. desember 2024

Tid: 09:00 - 13:00

Faglig kontakt: Magnus Jahre

Møter i eksamenslokalet: Ja. Jeg begynner å gå kl 09:30 og vil gå en runde gjennom lokalene.

Hjelpemiddelkode/Tillatte hjelpemidler: D: Ingen trykte eller håndskrevne hjelpemidler tillatt. Bestemt, enkel kalkulator tillatt.

ANNEN INFORMASJON:

Les oppgavene nøye og gjør dine egne antagelser. Presiser i besvarelsen hvilke forutsetninger du har lagt til grunn i tolkning/avgrensing av oppgaven.

Faglig kontaktperson skal kun kontaktes dersom det er direkte feil eller mangler i oppgavesettet. Henvend deg til en eksamensvakt hvis du mistenker feil og mangler. Noter spørsmålet ditt på forhånd.

FAGSPESIFIKK INFORMASJON

Ingen håndtegninger: Denne eksamenen tillater ikke bruk av håndtegninger. Har du likevel fått utdelt skanne-ark, er dette en feil. **Arkene vil ikke bli akseptert for innlevering, og de vil derfor heller ikke sendes til sensur.**

Vekting av oppgavene: Det gis inntil 100 poeng på denne eksamenen, og det er oppgitt hvor mange poeng som kan oppnås på hver deloppgave. Poengsummen til en deloppgave gir en indikasjon på forventet tidsbruk på deloppgaven.

Varslinger: Eventuelle beskjeder under eksamen (f.eks. ved feil i oppgavesettet), sendes ut via varslinger i Inspira. Et varsel vil dukke opp som en dialogboks på skjermen. Du kan finne igjen varselet ved å klikke på bjella øverst til høyre.

Trekk fra/avbrutt eksamen: Dersom du ønsker å levere blankt/avbryte eksamen, gå til "hamburgermenyen" i øvre høyre hjørne og velg «Lever blankt». Dette kan ikke angres selv om prøven fremdeles er åpen.

Tilgang til besvarelse: Etter eksamen finner du besvarelsen din under tidligere prøver i Inspira. Merk at det kan ta én virkedag før eventuelle håndtegninger vil være tilgjengelige under «tidligere prøver».

1.2 Koding av instruksjoner

funct7	rs2	rs1	funct3	rd	opcode
7 bits	5 bits	5 bits	3 bits	5 bits	7 bits

immediate	rs1	funct3	rd	opcode
12 bits	5 bits	3 bits	5 bits	7 bits

Hva blir feltverdiene under i binær for instruksjonen *add x1, x2, x3*? (3p)

- rs1:
- rs2:
- rd:

Hva blir feltverdiene under i binær for instruksjonen *addi x5, x4, -5*? (3p)

- rs1:
- rd:
- immediate:

Hva blir feltverdiene under i binær for instruksjonen *lw x7, 8(x6)*? (3p)

- rs1:
- rd:
- immediate:

Maks poeng: 9

1.3 Instruksjonsforståelse

Hvordan regner lw x7, 8(x6) ut adressen som skal lastes fra minnet? (2p)

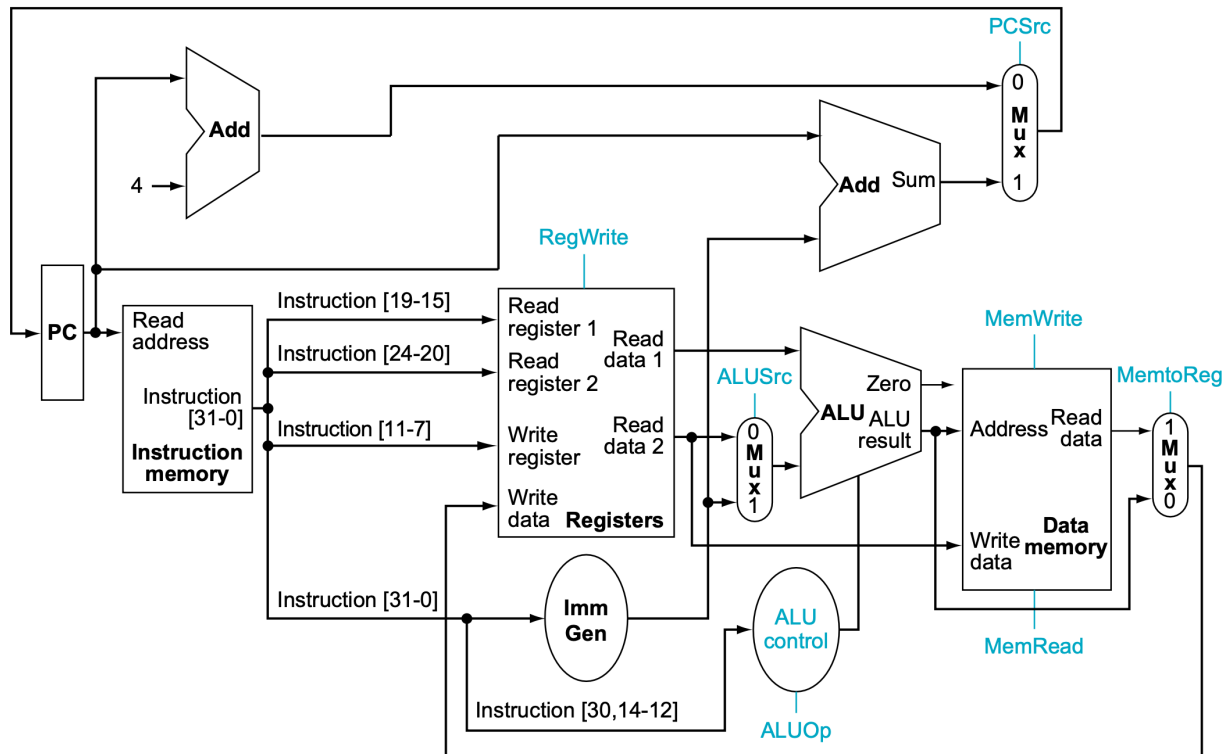
Skriv ditt svar her

Hvis du ønsker, kan du forklare hvordan du kom frem til svarene i oppgave 1.2 i tekstboksen under.

Skriv ditt svar her

Maks poeng: 2

2.1 Kontrollsignaler



Instruction opcode	ALUOp	Operation	Funct7 field	Funct3 field	Desired ALU action	ALU control input
lw	00	load word	XXXXXXX	XXX	add	0010
sw	00	store word	XXXXXXX	XXX	add	0010
beq	01	branch if equal	XXXXXXX	XXX	subtract	0110
R-type	10	add	0000000	000	add	0010
R-type	10	sub	0100000	000	subtract	0110
R-type	10	and	0000000	111	AND	0000
R-type	10	or	0000000	110	OR	0001

Denne oppgaven omhandler enkeltsykelprosessoren vist i figuren over. Et utdrag fra sannhetstabellen til enheten «ALU control» er også oppgitt.

Enkeltsykelprosessoren utfører instruksjonen *add x1, x2, x3*. Hvilke verdier har kontrollsignalene? (3p)

Skriv ditt svar her

Enkeltsykelprosessoren utfører instruksjonen *sw x7, 8(x6)*. Hvilke verdier har kontrollsignalene? (3p)

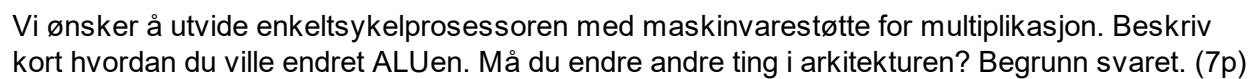
Skriv ditt svar her

Maks poeng: 6

Maks poeng: 6

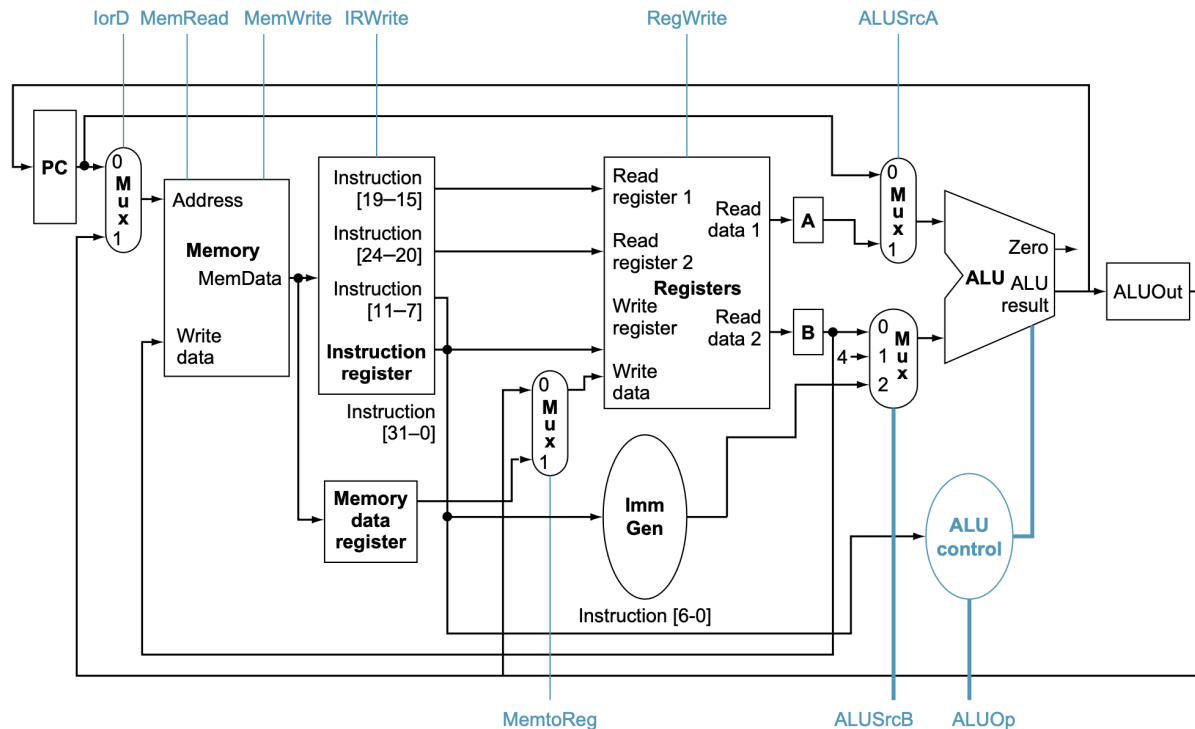
9/24

Maks poeng: 8

[illegible]

Maks poeng: 7

3 Flersykelprosessor



Denne oppgaven tar utgangspunkt i flersykelprosessen vist over.

Nevn en fordel med en flersykelprosessor sammenliknet med en enkeltsykelprosessor. (3p)

Skriv ditt svar her

Utføringen av en addisjonsinstruksjon vil ta 4 klokkesykler på denne arkitekturen, og vi er nå i den tredje klokkesyklen av instruksjonen *add x1, x2, x4* sin utføring.

Oppgi verdiene til alle kontrollsignalene. (Du kan se bort fra signalet «ALUOp»). (5p)

Skriv ditt svar her

Hvilke verdier er lagret i prosessorens interne registre ved slutten av klokkesyklen (altså i PC, «Instruction register», «Memory data register», A, B, og ALUOut)? (5p)

Skriv ditt svar her

Maks poeng: 13

4.1 Avhengigheter

```
1      addi x1, x0, 0
2      addi x2, x0, 5
3      addi x3, x0, 0
4      lui x3, 0x10000
5      addi x4, x0, 0
6
7      merkelapp:
8          lw x7, 0(x3)
9          add x4, x4, x7
10         addi x3, x3, 4
11         addi x1, x1, 1
12         ble x1, x2, merkelapp
13
```

Denne oppgaven tar utgangspunkt i programmet over. Hvilke dataavhengigheter er det i programmet? Oppgi avhengighetene ved å referere til linjenumrene til instruksjonene. (9p)

Skriv ditt svar her

Format | **B** | *I* | U | $\times_2$ | $\times^2$ | $\mathcal{I}_x$ | | | | | | | | | | | | Σ |

Words: 0

Maks poeng: 9



Skriv ditt svar her

17/24

Words: 0

Maks poeng: 3



Skriv ditt svar her

19/24

Skriv ditt svar her

Maks poeng: 8

5.1 Lokalitet

Forklar de to typene lokalitet og gi et eksempel på hvorfor hver av dem oppstår. (4p)

Skriv ditt svar her

Format

B

I

U

x_e

x^2

I_x





































Words: 0

Maks poeng: 4

5.2 Hurtigbuffer

Du skal lage et direkteilordnet hurtigbuffer som kan lagre 32 KiB med data til en datamaskin der minneadressene er 32 bit. Blokkstørrelsen skal være på 32 bit, og vi henter ut en blokk av gangen. Hvordan vil du dele inn adressen? Begrunn svaret. (6p)

Skriv ditt svar her

Format

B *I* U $\times_2$ $\times^2$ $\frac{\square}{\square}$

Words: 0

Maks poeng: 6

5.3 Hurtigbufferberegning

Hurtigbufferet i forrige oppgave integreres som nivå-1 hurtigbuffer i datamaskinen. Det gir en treffrate på 80% for Program A og en trefflatens på 1 klokkesykel. Det tar imidlertid 100 klokkesykler å aksessere minnet, så du foreslår å legge til et nivå-2 hurtigbuffer med 60% treffrate for Program A og 10 klokkesykler trefflatens. Estimer den gjennomsnittlige minneaksesstiden til din løsning. (4p)

Skriv ditt svar her

Format

B *I* U $\times_2$ $\times^2$ I_x

Words: 0

Maks poeng: 4

5.4 Bedre ytelse

Sjefen din er fortsatt ikke overbevist om at det er lurt å legge til et nivå-2 hurtigbuffer i arkitekturen. Du finner ut at Program A bruker 30% av kjøretiden sin til å vente på minnet. Hvilken ytelsesforbedring ("speed-up") forventer du Program A vil få med din løsning (to nivåer med hurtigbuffer) sammenliknet med sjefens løsning (ett nivå med hurtigbuffer)? (4p)

Skriv ditt svar her

Format

B *I* U $\times_2$ $\times^2$ $\mathcal{I}_x$

Words: 0

Maks poeng: 4

5.5 Virtuelt minne

Hva er virtuelt minne? Nevn en fordel med virtuelt minne og forklar kort hvordan datamaskiner får adresseoversettelse til å gå fort. (5p)

Skriv ditt svar her

Format

B

I

U

x_2

x^2

I_x



































Words: 0

Maks poeng: 5

i RISC-V Green Card

RISC-V Green Card vises til venstre.

i Engelsk-norsk ordliste

Ordlisten vises/synast til venstre.

Document 2

Attached



Base Integer Instructions: RV32I, RV64I, and RV128I						RV Privileged Instructions							
Category		Name	Fmt	RV32I Base		+RV{64,128}		Category		Name	RV mnemonic		
Loads	Load Byte	I	LB	rd,rs1,imm		L{D Q}	rd,rs1,imm	CSR Access	Atomic R/W	CSRRW	rd,csr,rs1		
	Load Halfword	I	LH	rd,rs1,imm			rd,csr,rs1						
	Load Word	I	LW	rd,rs1,imm			rd,csr,rs1						
	Load Byte Unsigned	I	LBU	rd,rs1,imm			rd,csr,imm						
	Load Half Unsigned	I	LHU	rd,rs1,imm			rd,csr,imm						
Stores	Store Byte	S	SB	rs1,rs2,imm		S{D Q}	rs1,rs2,imm	Atomic Read & Clear Bit Imm	CSRRCI	rd,csr,imm			
	Store Halfword	S	SH	rs1,rs2,imm									
	Store Word	S	SW	rs1,rs2,imm									
Shifts	Shift Left	R	SLL	rd,rs1,rs2		SLL{W D}	rd,rs1,rs2	Change Level	Env. Call	ECALL			
	Shift Left Immediate	I	SLLI	rd,rs1,shamt		SLLI{W D}	rd,rs1,shamt		Environment Breakpoint	EBREAK			
	Shift Right	R	SRL	rd,rs1,rs2		SRL{W D}	rd,rs1,rs2		Environment Return	ERET			
	Shift Right Immediate	I	SRLI	rd,rs1,shamt		SRLI{W D}	rd,rs1,shamt	Trap Redirect to Supervisor	MRTS				
	Shift Right Arithmetic	R	SRA	rd,rs1,rs2		SRA{W D}	rd,rs1,rs2		Redirect Trap to Hypervisor	MRTH			
	Shift Right Arith Imm	I	SRAI	rd,rs1,shamt		SRAI{W D}	rd,rs1,shamt		Hypervisor Trap to Supervisor	HRTS			
Arithmetic	ADD	R	ADD	rd,rs1,rs2		ADD{W D}	rd,rs1,rs2	Interrupt	Wait for Interrupt	WFI			
	ADD Immediate	I	ADDI	rd,rs1,imm		ADDI{W D}	rd,rs1,imm		MMU	Supervisor FENCE	SFENCE.VM rs1		
	SUBtract	R	SUB	rd,rs1,rs2		SUB{W D}	rd,rs1,rs2						
	Load Upper Imm	U	LUI	rd,imm		Optional Compressed (16-bit) Instruction Extension: RVC							
	Add Upper Imm to PC	U	AUIPC	rd,imm		Category	Name	Fmt	RVC		RVI equivalent		
Logical	XOR	R	XOR	rd,rs1,rs2		Loads	Load Word	CL	C.LW	rd',rs1',imm	LW rd',rs1',imm*4		
	XOR Immediate	I	XORI	rd,rs1,imm			Load Word SP	CI	C.LWSP	rd,imm	LW rd,sp,imm*4		
	OR	R	OR	rd,rs1,rs2			Load Double	CL	C.LD	rd',rs1',imm	LD rd',rs1',imm*8		
	OR Immediate	I	ORI	rd,rs1,imm			Load Double SP	CI	C.LDSP	rd,imm	LD rd,sp,imm*8		
	AND	R	AND	rd,rs1,rs2			Load Quad	CL	C.LQ	rd',rs1',imm	LQ rd',rs1',imm*16		
	AND Immediate	I	ANDI	rd,rs1,imm			Load Quad SP	CI	C.LQSP	rd,imm	LQ rd,sp,imm*16		
Compare	Set <	R	SLT	rd,rs1,rs2		Stores	Store Word	CS	C.SW	rs1',rs2',imm	SW rs1',rs2',imm*4		
	Set < Immediate	I	SLTI	rd,rs1,imm			Store Word SP	CSS	C.SWSP	rs2,imm	SW rs2,sp,imm*4		
	Set < Unsigned	R	SLTU	rd,rs1,rs2			Store Double	CS	C.SD	rs1',rs2',imm	SD rs1',rs2',imm*8		
	Set < Imm Unsigned	I	SLTIU	rd,rs1,imm			Store Double SP	CSS	C.SDSP	rs2,imm	SD rs2,sp,imm*8		
	Branches	Branch =	SB	BEQ	rs1,rs2,imm		Store Quad	CS	C.SQ	rs1',rs2',imm	SQ rs1',rs2',imm*16		
Branch ≠		SB	BNE	rs1,rs2,imm		Store Quad SP	CSS	C.SQSP	rs2,imm	SQ rs2,sp,imm*16			
Branch <		SB	BLT	rs1,rs2,imm		Arithmetic	ADD	CR	C.ADD	rd,rs1	ADD rd,rd,rs1		
Branch ≥		SB	BGE	rs1,rs2,imm			ADD Word	CR	C.ADDW	rd,rs1	ADDW rd,rd,imm		
Branch < Unsigned		SB	BLTU	rs1,rs2,imm			ADD Immediate	CI	C.ADDI	rd,imm	ADDI rd,rd,imm		
Branch ≥ Unsigned		SB	BGEU	rs1,rs2,imm			ADD Word Imm	CI	C.ADDIW	rd,imm	ADDIW rd,rd,imm		
Jump & Link	J&L	UJ	JAL	rd,imm			ADD SP Imm * 16	CI	C.ADDI16SP	x0,imm	ADDI sp,sp,imm*16		
	Jump & Link Register	UJ	JALR	rd,rs1,imm			ADD SP Imm * 4	CIW	C.ADDI4SPN	rd',imm	ADDI rd',sp,imm*4		
Synch	Synch thread	I	FENCE			Load Immediate	CI	C.LI	rd,imm	ADDI rd,x0,imm			
	Synch Instr & Data	I	FENCE.I			Load Upper Imm	CI	C.LUI	rd,imm	LUI rd,imm			
System	System CALL	I	SCALL			MoVe	CR	C.MV	rd,rs1	ADD rd,rs1,x0			
	System BREAK	I	SBREAK			SUB	CR	C.SUB	rd,rs1	SUB rd,rd,rs1			
Counters	ReaD CYCLE	I	RDCYCLE	rd		Shifts	Shift Left Imm	CI	C.SLLI	rd,imm	SLLI rd,rd,imm		
	ReaD CYCLE upper Half	I	RDCYCLEH	rd		Branches	Branch=0	CB	C.BEQZ	rs1',imm	BEQ rs1',x0,imm		
	ReaD TIME	I	RDTIME	rd			Branch≠0	CB	C.BNEZ	rs1',imm	BNE rs1',x0,imm		
	ReaD TIME upper Half	I	RDTIMEH	rd		Jump	Jump	CJ	C.J	imm	JAL x0,imm		
	ReaD INSTR RETired	I	RDINSTRET	rd			Jump Register	CR	C.JR	rd,rs1	JALR x0,rs1,0		
	ReaD INSTR upper Half	I	RDINSTRETH	rd		Jump & Link	J&L	CJ	C.JAL	imm	JAL ra,imm		
		Jump & Link Register	CR	C.JALR	rs1		Jump & Link Register	CR	C.JALR	rs1		JALR ra,rs1,0	
		System Env. BREAK	CI	C.EBREAK			System	Env. BREAK	CI	C.EBREAK			EBREAK

32-bit Instruction Formats																16-bit (RVC) Instruction Formats																				
	31	30	25	24	21	20	19	15	14	12	11	8	7	6	0	CR	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0				
R I S S B U J	funct7				rs2		rs1		funct3		rd		opcode				CS CIW CL CS CB CB	funct4				rd/rs1				rs2				op						
	imm[11:0]						rs1		funct3		rd		opcode					funct3	imm				rd/rs1				imm				op					
	imm[11:5]				rs2		rs1		funct3		imm[4:0]		opcode					funct3	imm								rs2				op					
	imm[12]		imm[10:5]		rs2		rs1		funct3		imm[4:1]		imm[11]		opcode				funct3	imm				rs1'		imm		rd'		op						
	imm[31:12]												rd					opcode				funct3	imm				rs1'		imm		rs2'		op			
	imm[20]				imm[10:1]				imm[11]		imm[19:12]				rd				opcode				funct3	offset				rs1'		offset		op				
													rd				opcode				funct3	jump target								op						

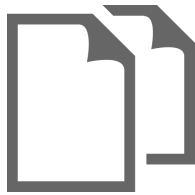
RISC-V Integer Base (RV32I/64I/128I), privileged, and optional compressed extension (RVC). Registers x1-x31 and the pc are 32 bits wide in RV32I, 64 in RV64I, and 128 in RV128I (x0=0). RV64I/128I add 10 instructions for the wider formats. The RVI base of <50 classic integer RISC instructions is required. Every 16-bit RVC instruction matches an existing 32-bit RVI instruction. See risc.org.

Optional Multiply-Divide Instruction Extension: RVM								
Category	Name	Fmt	RV32M (Multiply-Divide)		+RV{64,128}			
Multiply	MULTiply	R	MUL	rd,rs1,rs2	MUL{W D}	rd,rs1,rs2		
	MULTiply upper Half	R	MULH	rd,rs1,rs2				
	MULTiply Half Sign/Uns	R	MULHSU	rd,rs1,rs2				
	MULTiply upper Half Uns	R	MULHU	rd,rs1,rs2				
Divide	DIVide	R	DIV	rd,rs1,rs2	DIV{W D}	rd,rs1,rs2		
	DIVide Unsigned	R	DIVU	rd,rs1,rs2				
Remainder	REMAinder	R	REM	rd,rs1,rs2	REM{W D}	rd,rs1,rs2		
	REMAinder Unsigned	R	REMU	rd,rs1,rs2	REMU{W D}	rd,rs1,rs2		
Optional Atomic Instruction Extension: RVA								
Category	Name	Fmt	RV32A (Atomic)		+RV{64,128}			
Load	Load Reserved	R	LR.W	rd,rs1	LR.{D Q}	rd,rs1		
Store	Store Conditional	R	SC.W	rd,rs1,rs2	SC.{D Q}	rd,rs1,rs2		
Swap	SWAP	R	AMOSWAP.W	rd,rs1,rs2	AMOSWAP.{D Q}	rd,rs1,rs2		
Add	ADD	R	AMOADD.W	rd,rs1,rs2	AMOADD.{D Q}	rd,rs1,rs2		
Logical	XOR	R	AMOXOR.W	rd,rs1,rs2	AMOXOR.{D Q}	rd,rs1,rs2		
	AND	R	AMOAND.W	rd,rs1,rs2	AMOAND.{D Q}	rd,rs1,rs2		
	OR	R	AMOOR.W	rd,rs1,rs2	AMOOR.{D Q}	rd,rs1,rs2		
Min/Max	MINimum	R	AMOMIN.W	rd,rs1,rs2	AMOMIN.{D Q}	rd,rs1,rs2		
	MAXimum	R	AMOMAX.W	rd,rs1,rs2	AMOMAX.{D Q}	rd,rs1,rs2		
	MINimum Unsigned	R	AMOMINU.W	rd,rs1,rs2	AMOMINU.{D Q}	rd,rs1,rs2		
	MAXimum Unsigned	R	AMOMAXU.W	rd,rs1,rs2	AMOMAXU.{D Q}	rd,rs1,rs2		
Three Optional Floating-Point Instruction Extensions: RVF, RVD, & RVQ								
Category	Name	Fmt	RV32{F D Q} (HP/SP,DP,QP FI Pt)		+RV{64,128}			
Move	Move from Integer	R	FMV.{H S}.X	rd,rs1	FMV.{D Q}.X	rd,rs1		
	Move to Integer	R	FMV.X.{H S}	rd,rs1	FMV.X.{D Q}	rd,rs1		
Convert	Convert from Int	R	FCVT.{H S D Q}.W	rd,rs1	FCVT.{H S D Q}.{L T}	rd,rs1		
	Convert from Int Unsigned	R	FCVT.{H S D Q}.WU	rd,rs1	FCVT.{H S D Q}.{L T}U	rd,rs1		
	Convert to Int	R	FCVT.W.{H S D Q}	rd,rs1	FCVT.{L T}.{H S D Q}	rd,rs1		
	Convert to Int Unsigned	R	FCVT.WU.{H S D Q}	rd,rs1	FCVT.{L T}U.{H S D Q}	rd,rs1		
RISC-V Calling Convention								
				Register	ABI Name	Saver	Description	
Arithmetic	Load	I	FL{W,D,Q}	rd,rs1,imm				
	Store	S	FS{W,D,Q}	rs1,rs2,imm				
	ADD	R	FADD.{S D Q}	rd,rs1,rs2	x0	zero	---	Hard-wired zero
	SUBtract	R	FSUB.{S D Q}	rd,rs1,rs2	x1	ra	Caller	Return address
	MULTiply	R	FMUL.{S D Q}	rd,rs1,rs2	x2	sp	Callee	Stack pointer
Mul-Add	DIVide	R	FDIV.{S D Q}	rd,rs1,rs2	x3	gp	---	Global pointer
	SQuare RooT	R	FSQRT.{S D Q}	rd,rs1	x4	tp	---	Thread pointer
	MULTiply-ADD	R	FMADD.{S D Q}	rd,rs1,rs2,rs3	x5-7	t0-2	Caller	Temporaries
	MULTiply-SUBtract	R	FMSUB.{S D Q}	rd,rs1,rs2,rs3	x8	s0/fp	Callee	Saved register/frame pointer
	Negative Multiply-SUBtract	R	FNMSUB.{S D Q}	rd,rs1,rs2,rs3	x9	s1	Callee	Saved register
Sign Inject	Negative Multiply-ADD	R	FNMADD.{S D Q}	rd,rs1,rs2,rs3	x10-11	a0-1	Caller	Function arguments/return values
	SIGN source	R	FSGNJ.{S D Q}	rd,rs1,rs2	x12-17	a2-7	Caller	Function arguments
	Negative SIGN source	R	FSGNJN.{S D Q}	rd,rs1,rs2	x18-27	s2-11	Callee	Saved registers
Min/Max	Xor SIGN source	R	FSGNJX.{S D Q}	rd,rs1,rs2	x28-31	t3-t6	Caller	Temporaries
	MINimum	R	FMIN.{S D Q}	rd,rs1,rs2	f0-7	ft0-7	Caller	FP temporaries
Compare	MAXimum	R	FMAX.{S D Q}	rd,rs1,rs2	f8-9	fs0-1	Callee	FP saved registers
	Compare Float =	R	FEQ.{S D Q}	rd,rs1,rs2	f10-11	fa0-1	Caller	FP arguments/return values
	Compare Float <	R	FLT.{S D Q}	rd,rs1,rs2	f12-17	fa2-7	Caller	FP arguments
Categorization	Compare Float ≤	R	FLE.{S D Q}	rd,rs1,rs2	f18-27	fs2-11	Callee	FP saved registers
	Classify Type	R	FCLASS.{S D Q}	rd,rs1	f28-31	ft8-11	Caller	FP temporaries
Configuration	Read Status	R	FRCSR	rd				
	Read Rounding Mode	R	FRRM	rd				
	Read Flags	R	FRFLAGS	rd				
	Swap Status Reg	R	FSCSR	rd,rs1				
	Swap Rounding Mode	R	FSRM	rd,rs1				
	Swap Flags	R	FSFLAGS	rd,rs1				
	Swap Rounding Mode Imm	I	FSRMI	rd,imm				
	Swap Flags Imm	I	FSFLAGSI	rd,imm				

RISC-V calling convention and five optional extensions: 10 multiply-divide instructions (RV32M); 11 optional atomic instructions (RV32A); and 25 floating-point instructions each for single-, double-, and quadruple-precision (RV32F, RV32D, RV32Q). The latter add registers f0-f31, whose width matches the widest precision, and a floating-point control and status register fcsr. Each larger address adds some instructions: 4 for RVM, 11 for RVA, and 6 each for RVF/D/Q. Using regex notation, { } means set, so L{D|Q} is both LD and LQ. See riscv.org. (8/21/15 revision)

Document 3

Attached



Engelsk-norsk ordliste for TDT4160

Sist oppdatert 28/11-24

Engelsk

Abstraction layer
Address space
Address translation
Addressing mode
Analog
Arithmetic intensity
Array
Asserted
Base address
Benchmark
Benchmark suite
Branch
Branch instruction
Branch prediction
Branch, conditional
Branch, unconditional
Bus
Cache
Cache block
Cache line
Carry
Central Processing Unit (CPU)
Circuit
Clock cycle
Clock frequency
Clock period
Coherency
Compiler
Concurrent
Consistency
Constant
Context switch
Control
Control hazard
Cycle time
Data hazard
Datapath
Deasserted
Dependency
Digital
Direct-mapped cache
Distributed memory
Domain-Specific Accelerator (DSE)
Dynamic

Norsk

Abstraksjonslag
Adresserom
Adresseoversettelse
Adressemodus
Analog
Operasjonsintensitet
Tabell
Satt
Grunnadresse
Testprogram
Testprogramsamling
Hopp
Forgreningsinstruksjon
Forgreningsprediksjon
Hopp, betinget
Hopp, ubetinget
Buss
Hurtigbuffer
Blokk (i hurtigbufferet)
Linje (i hurtigbufferet)
Mente
Prosesor eller CPU
Krets
Klokkesykel
Klokkefrekvens
Klokkeperiode
Koherens
Kompilator
Samtidig
Konsistens
Konstant
Kontekstbytte
Kontrollenhet
Kontrollfare
Sykeltid
Datafare
Datasti
Ikke satt
Avhengighet
Digital
Direktetilordnet hurtigbuffer
Distribuert minne
Domenespesifik akselerator eller DSE
Dynamisk

Embedded system	Innvevd system
Energy	Energi
Energy consumption	Energiforbruk
Exception	Unntak
Execution time	Kjøretid
Flip-flop	Vippe
Flush	Fjerne
Fully associative cache	Fullassosiativt hurtigbuffer
Gate	Port
Graphics Processing Unit (GPU)	Grafikkprosessor eller GPU
Hardware	Maskinvare
Hazard	Fare
Immediate	Konstant
Instruction Set Architecture (ISA)	Instruksjonssett eller ISA
Instruction-Level Parallelism (ILP)	Instruksjonsnivåparallellitet
Integrated circuit	Integrert krets
Interrupt	Avbrudd
Jump	Hopp
Latch	Lås
Load-use hazard	Les-bruk fare
Locality	Lokalitet
Memory	Minne
Multi-core processor	Flerkjerneprosessor
Multiprocessor	Multiprosessor
Non-volatile	Ikke-volatil
Operand	Operand
Operating system	Operativsystem
Operational intensity	Operasjonsintensitet
Overflow	Overflyt
Page fault	Sidefeil
Parallel	Parallell
Parallel processing program	Parallelt program
Performance	Ytelse
Pipeline	Samlebånd
Pipeline bubble	Hull i samlebåndet
Power	Effekt
Power consumption	Effektforbruk
Privileged instruction	Privilegert instruksjon
Processor	Prosessor
Program	Dataprogram (eller bare program)
Program Counter (PC)	Programteller eller PC
Protection	Beskyttelse
Register	Register
Serial	Seriell
Set-associative cache	Settassosiativt hurtigbuffer
Shared memory	Delt minne
Sign bit	Fortegnsbit
Sign extension	Fortegnsutvidelse

Silicon	Silisium
Software	Programvare
Spatial locality	Lokalitet i rom
Stall	Stans
Static	Statisk
Stored-program concept	Prinsippet om lagrede program
Strong scaling	Sterk skalering
Structural hazard	Strukturfare
System call	Systemkall
System mode	Systemmodus
Temporal locality	Lokalitet i tid
Throughput	Gjennomstrømning
User mode	Brukermodus
Virtual memory	Virtuelt minne
Volatile	Volatil
Way	Vei (som i et 4-veis settassosiativt hurtigbuffer)
Weak scaling	Svak skalering
Word	Ord (en 32-bits verdi)
Workload	Arbeidslast